

Programming II

Object Oriented Programming (OOP)

Introduction to OOP

Object and Class

What is OOP?

- Object-oriented programming (OOP) is a programming paradigm based on the concept of “**objects**” .
- A programming paradigm is a style of programming, a way of thinking about software construction.
- A programming paradigm does not refer to a specific language but rather to a way to build a program or a methodology to apply.
- Some languages make it easy to write in some paradigms but not others.
- Some programming languages allow the programmer to apply more than one paradigm.

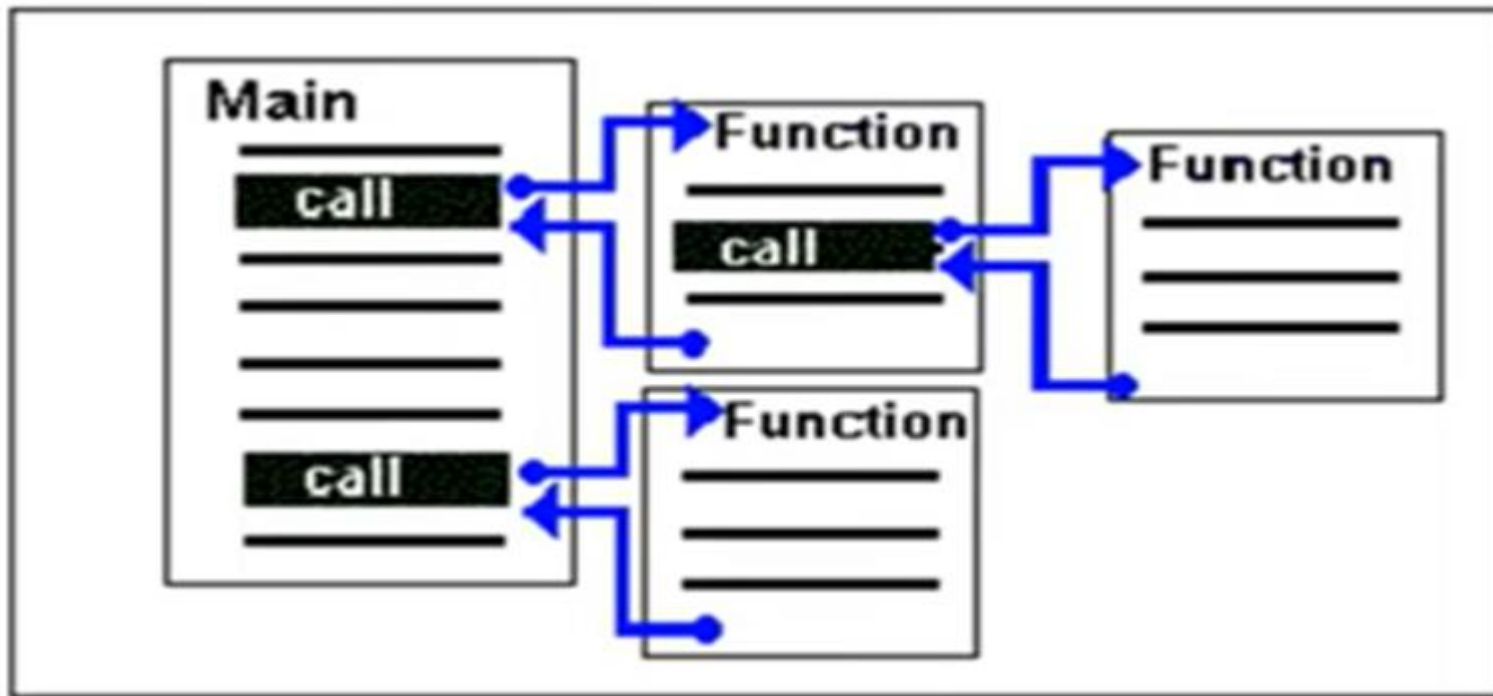
Programming Paradigms:

- **Procedural:** Basic, Pascal.
- **Object-oriented:** C++, Java
- **Declarative:** Prolog.
- **Event-driven:** Visual Basic, C#

Examples on Programming Paradigms:

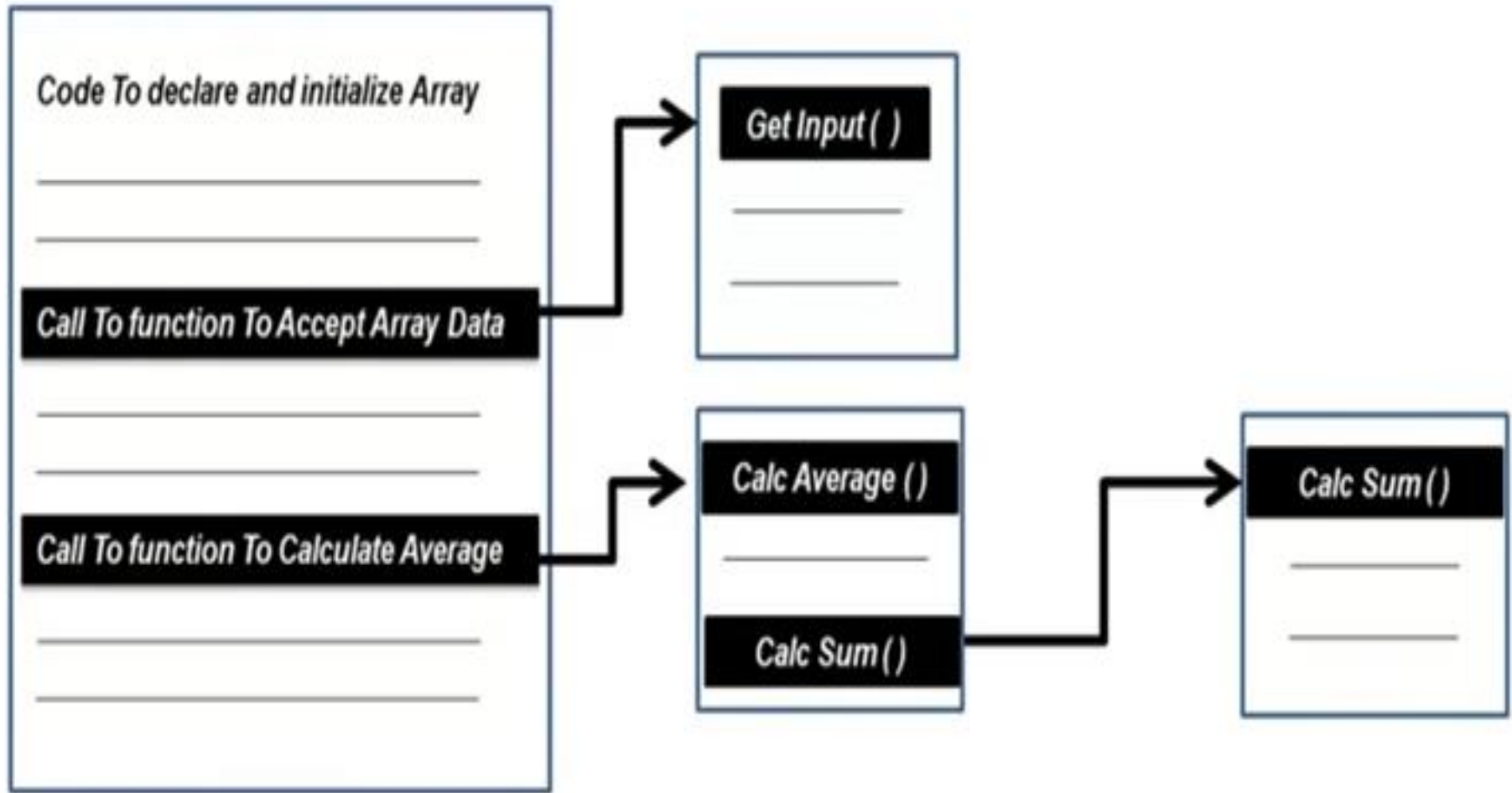
Procedural Programming (PP):

- It is about writing a list of instructions to tell the computer what to do step by step.
- It relies on procedures.



Procedural Programming Example:

- Program to calculate average of array items.



Object Oriented Programming (OOP):

- Object-oriented programming (OOP) is a programming paradigm based on the concept of “**objects**” .

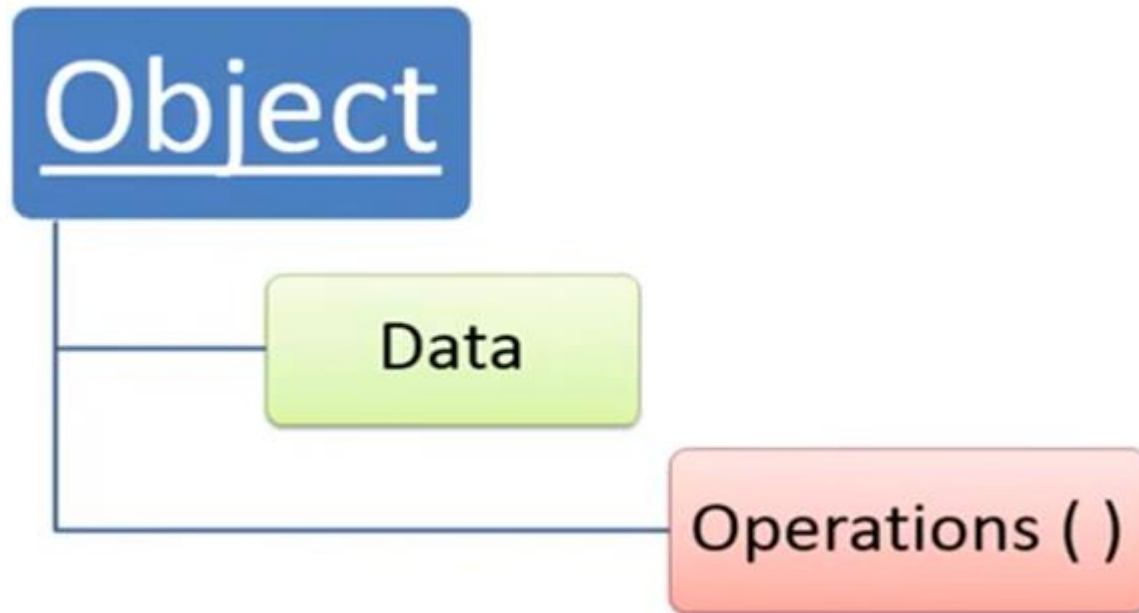
Example:

Objects in College Management Program:

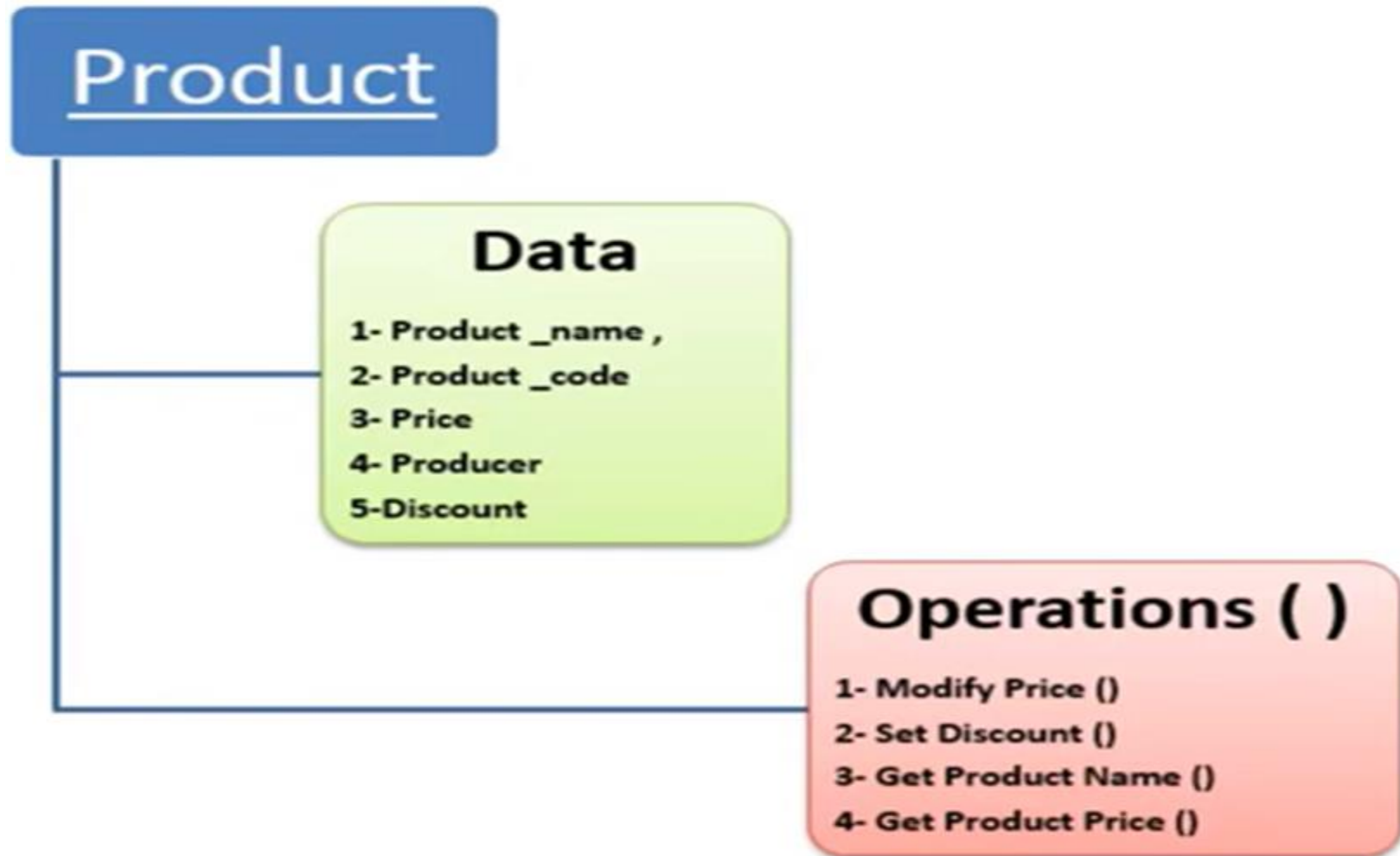


Object:

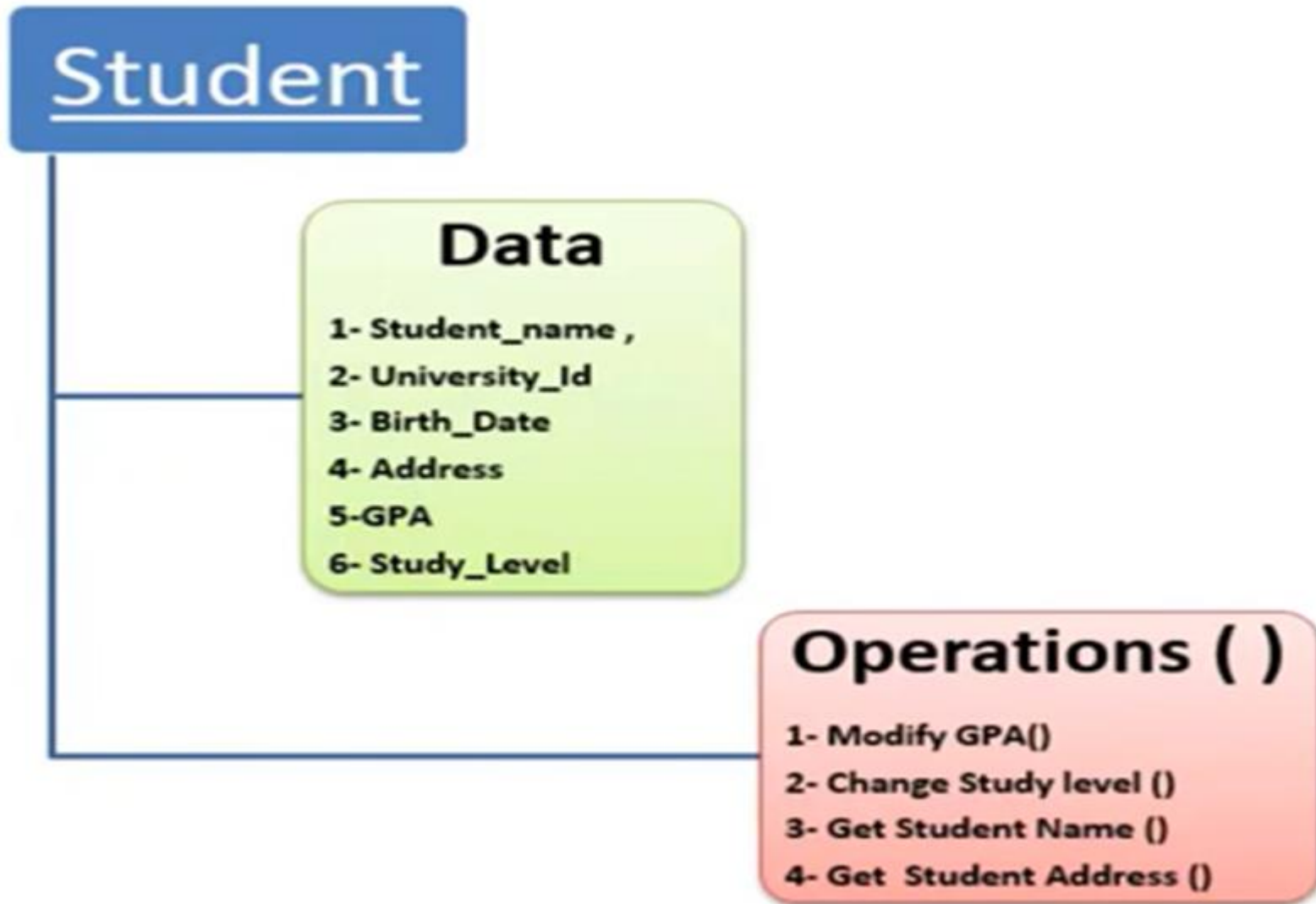
- Object consists of (**Data + Operations**)



Examples:



Examples:



What is class? Why we need it?

Student 1

Data:

- 1- Student_name ,
- 2- University_Id
- 3- Birth_Date
- 4- Address
- 5-GPA
- 6- Study_Level

Operations ()

- 1- Modify GPA()
- 2- Change Study level ()
- 3- Get Student Name ()
- 4- Get Student Address ()

Student 2

Data:

- 1- Student_name ,
- 2- University_Id
- 3- Birth_Date
- 4- Address
- 6- Study_Level

Operations ()

- 1- Modify GPA()
- 2- Change Study level ()
- 4- Get Student Address ()

Student 3

Data:

- 1- Student_name ,
- 2- University_Id
- 5-GPA
- 6- Study_Level

Operations ()

- 1- Modify GPA()
- 2- Change Study level ()
- 3- Get Student Name ()
- 4- Get Student Address ()

What is class? Why we need it?

Class Student

Data:

- 1- Student_name ,
- 2- University_Id
- 3- Birth_Date
- 4- Address
- 5-GPA
- 6- Study_Level

Operations ()

- 1- Modify GPA()
- 2- Change Study level ()
- 3- Get Student Name ()
- 4- Get Student Address ()

Student 1

Data:

- 1- Student_name
- 2- University_Id
- 3- Birth_Date
- 4- Address
- 5-GPA
- 6- Study_Level

Operations ()

- 1- Modify GPA()
- 2- Change Study level ()
- 3- Get Student Name ()
- 4- Get Student Address ()

Student 3

Data:

- 1- Student_name
- 2- University_Id
- 3- Birth_Date
- 4- Address
- 5-GPA
- 6- Study_Level

Operations ()

- 1- Modify GPA()
- 2- Change Study level ()
- 3- Get Student Name ()
- 4- Get Student Address ()

Student 2

Data:

- 1- Student_name
- 2- University_Id
- 3- Birth_Date
- 4- Address
- 5-GPA
- 6- Study_Level

Operations ()

- 1- Modify GPA()
- 2- Change Study level ()
- 3- Get Student Name ()
- 4- Get Student Address ()

What is class? Why we need it?

Class Student

Data:

- 1- Student_name ,
- 2- University_Id
- 3- Birth_Date
- 4- Address
- 5-GPA
- 6- Study_Level

7- Email

Operations ()

- 1- Modify GPA()
- 2- Change Study level ()
- 3- Get Student Name ()
- 4- Get Student Address ()

Student 1

Data:

- 1- Student_name
- 2- University_Id
- 3- Birth_Date
- 4- Address
- 5-GPA
- 6- Study_Level

7- Email

Operations ()

- 1- Modify GPA()
- 2- Change Study level ()
- 3- Get Student Name ()
- 4- Get Student Address ()

Student 3

Data:

- 1- Student_name
- 2- University_Id
- 3- Birth_Date
- 4- Address
- 5-GPA
- 6- Study_Level

7- Email

Operations ()

- 1- Modify GPA()
- 2- Change Study level ()
- 3- Get Student Name ()
- 4- Get Student Address ()

Student 2

Data:

- 1- Student_name
- 2- University_Id
- 3- Birth_Date
- 4- Address
- 5-GPA
- 6- Study_Level

7- Email

Operations ()

- 1- Modify GPA()
- 2- Change Study level ()
- 3- Get Student Name ()
- 4- Get Student Address ()

What is class? Why we need it?

Class Student

Data:

- 1- Student_name ,
- 2- University_Id
- 3- Birth_Date
- 4- Address
- 5-GPA
- 6- Study_Level

7- Email

Operations ()

- 1- Modify GPA()
- 2- Change Study level ()
- 3- Get Student Name ()
- 4- Get Student Address ()

5- Print Student Info ()

Student 1

Data:

- 1- Student_name
- 2- University_Id
- 3- Birth_Date
- 4- Address
- 5-GPA
- 6- Study_Level

7- Email

Operations ()

- 1- Modify GPA()
- 2- Change Study level ()
- 3- Get Student Name ()
- 4- Get Student Address ()

5- Print Student Info ()

Student 2

Data:

- 1- Student_name
- 2- University_Id
- 3- Birth_Date
- 4- Address
- 5-GPA
- 6- Study_Level

7- Email

Operations ()

- 1- Modify GPA()
- 2- Change Study level ()
- 3- Get Student Name ()
- 4- Get Student Address ()

5- Print Student Info ()

Student 3

Data:

- 1- Student_name
- 2- University_Id
- 3- Birth_Date
- 4- Address
- 5-GPA
- 6- Study_Level

7- Email

Operations ()

- 1- Modify GPA()
- 2- Change Study level ()
- 3- Get Student Name ()
- 4- Get Student Address ()

5- Print Student Info ()

What is class? Why we need it?

Class Student

Data:

- 1- Student_name ,
- 2- University_Id
- 5-GPA
- 6- Study_Level

Operations ()

- 1- Modify GPA()
- 2- Change Study level ()
- 3- Get Student Name ()
- 4- Get Student GPA ()

Student 1

Data:

- | | |
|------------------|---------|
| 1- Student_name | = Ahmed |
| 2- University_Id | = 1050 |
| 5-GPA | =3.75 |
| 6- Study_Level | = 5 |

Operations ()

- 1- Modify GPA()
- 2- Change Study level ()
- 3- Get Student Name ()
- 4- Get Student GPA ()

Objects and Classes:

- Classes: where objects come from
 - A class is code that describes a particular type of object. It specifies the data that an object can hold (the object's fields), and the actions that an object can perform (the object's methods).
- When a program is running, it can use the class to create, in memory, as many objects of a specific type as needed.
- Each object that is created from a class is called an **instance** of the class.

Objects and Classes:

Example:

This expression creates a Scanner object in memory.

```
Scanner input = new Scanner(System.in);
```

The object's memory address
is assigned to the Input
variable.



Objects and Classes:

Example:

This expression creates a
Random object in memory.

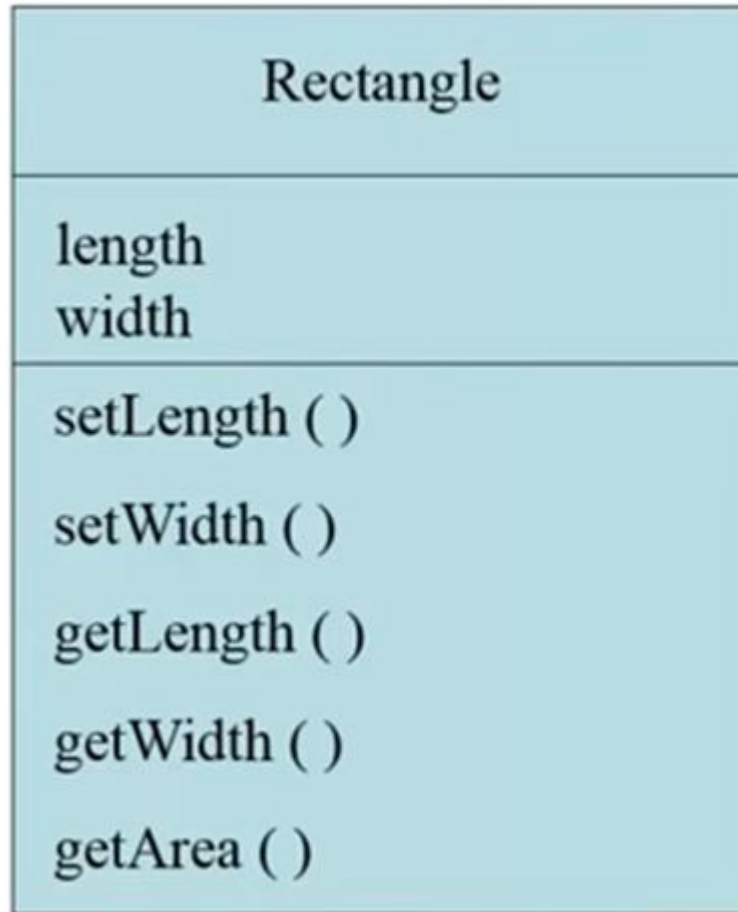
```
Random rand = new Random();
```

The object's memory address is
assigned to the `rand` variable.



Writing a class, step by step:

- A Rectangle class will have the following fields:



- UML (**U**nified **M**odeling **L**anguage) Class Diagram

Writing the code:

```
public class Rectangle
{
    private double length;
    private double width;
}
```

Rectangle
length width
setLength() setWidth() getLength() getWidth() getArea()

Access Modifiers:

- An access modifier is a java keyword that indicates how a field or method can be accessed.

Public:

- When the public access modifier is applied to a class member (field or method inside the class), the member can be accessed by the code inside the class or outside.

Private:

- When the private access modifier is applied to a class member, the member cannot be accessed by the code outside the class. The member can be accessed only by methods are members of the same class.

Encapsulation (Data Hiding):

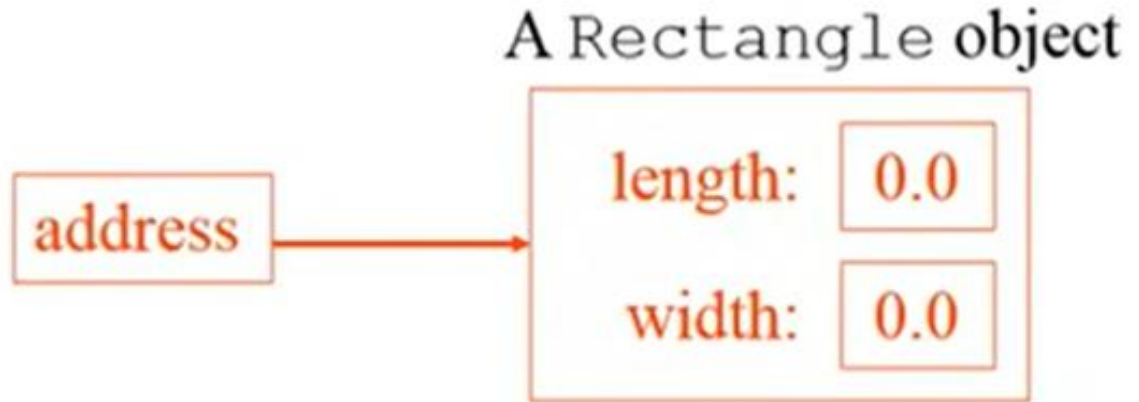
- An object hides its internal private fields from code that is outside the class that the object is an instance of.
- Only the class's methods may directly access and change the object's internal data.
- Code outside the class must use the class's public methods to operate on an object's private fields.
- Data hiding is important because classes are typically used as components in large software systems involving a team of programmers.

Example:

Creating a Rectangle object:

Rectangle r1 = new Rectangle ();

The **r1**
variable holds
the address of
the Rectangle
object.



Example:

```
public class Rectangle{  
    private double length;  
    public double width;  
}
```

```
public class Main  
{  
    public static void main(String[] args) {  
        Rectangle r1 = new Rectangle ();  
        r1.width = 10.5;  
        r1.length = 60;  
    }  
}
```

```
Main.java:14: error: length has private access in Rectangle  
        r1.length = 60;  
            ^
```

```
1 error
```

Example :

```
public class Main
{
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle ();
        r1.width = -7;
    }
}
```


Example :

```
public class Rectangle{
    private double length;
    private double width;

    public void setlength (double l)
    {
        length = l;
    }
    public void setwidth (double w)
    {
        width = w;
    }
}
```

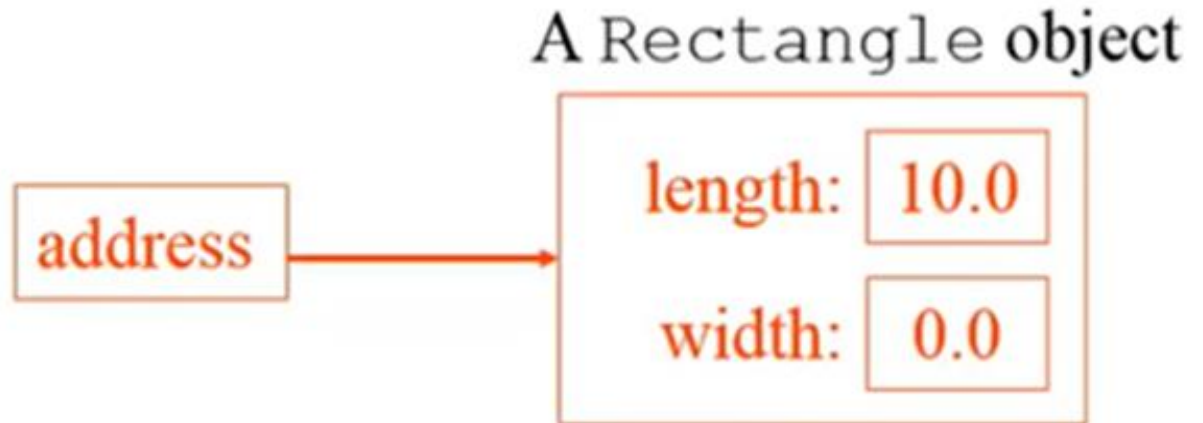
```
public class Main
{
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle ();
        r1.setlength(10);
        r1.setwidth(12.5);
    }
}
```

Calling the setlength Method:

`r1.setlength(10);`

- This is the state of the `r1` object after the `setlength` method executes.

The `r1` variable holds the address of the Rectangle object.



Example:

```
public class Rectangle{
    private double length;
    private double width;

    public void setlength (double l)
    {
        length = l;
    }
    public void setwidth (double w)
    {
        width = w;
    }
    public double getlength()
    {
        return length;
    }
    public double getwidth()
    {
        return width;
    }
    public double getarea()
    {
        return length*width;
    }
}
```

Rectangle
length width
setLength () setWidth () getLength () getWidth () getArea ()

Example:

```
public class Main
{
    public static void main(String[] args) {
        Rectangle r1 = new Rectangle ();
        r1.setlength(10);
        r1.setwidth(12.5);
        Rectangle r2 = new Rectangle ();
        r2.setlength(18);
        r2.setwidth (20);
        System.out.println (r1.getlength());
        System.out.println (r2.getarea());
    }
}
```

```
10.0
360.0
```

Setters (Mutators) and Getters (Accessors):

```
public class Rectangle  
{
```

```
    private double width;  
    private double length;
```

```
    public void setWidth(double w)  
    {  
        width = w;  
    }
```

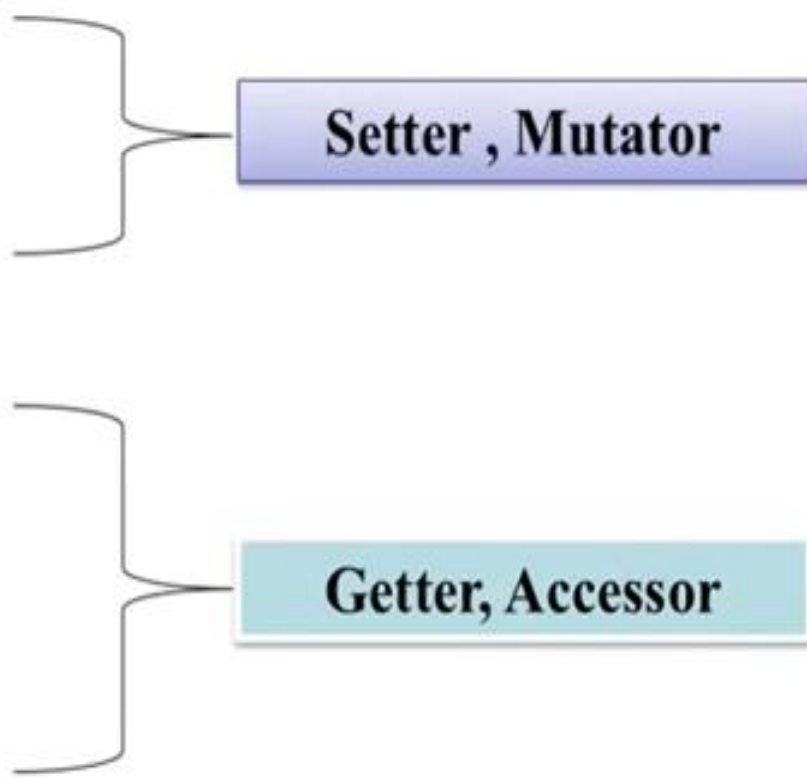
```
    public void setLength(double len)  
    {  
        length = len;  
    }
```

```
    public double getWidth()  
    {  
        return width;  
    }
```

```
    public double getLength()  
    {  
        return length;  
    }
```

```
    public double getArea()  
    {  
        return length * width;  
    }
```

```
}
```



Setter, Mutator

Getter, Accessor

Uninitialized Reference Variables:

- Reference variables can be declared without being initialized

Rectangle box;

box: reference variable (store address of the object)

- This statement does not create a Rectangle object, so it is an uninitialized reference variable.
- A reference variable must reference an object before it can be used, otherwise a compiler error will occur.

box = new Rectangle();



Rectangle box;

box = new Rectangle();

= Rectangle box = new Rectangle();

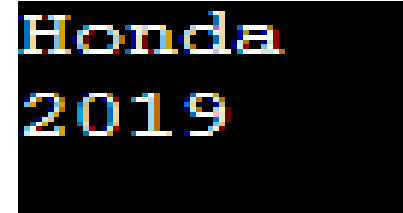
Example:

```
public class Car{  
    private String maker;  
    private int model;  
    public void setmaker (String m)  
    {    maker = m;  
    }  
    public void setmodel (int year)  
    { model = year;  
    }  
    public String getmaker()  
    { return maker;  
    }  
    public int getmodel()  
    { return model;  
    }  
}
```

Car
- maker - model
+ setmaker() + setmodel() + getmaker() + getmodel()

Example:

```
public class Main
{
    public static void main(String[] args) {
        Car c1;
        c1 = new Car();
        Car c2 = new Car();
        c1.setmaker("Honda");
        c1.setmodel(2016);
        c2.setmaker("Toyota");
        c2.setmodel (2019);
        System.out.println(c1.getmaker());
        System.out.println(c2.getmodel());
    }
}
```



Honda
2019

Example (Data Hiding)

```
public class Car{
    private String maker;
    private int model;
    public void setmaker (String m)
    {    if (m == "Toyota" || m== "Honda" || m=="Mercedes")
        maker = m;
        else
            System.out.println("invalid maker");
    }
    public void setmodel (int year)
    {    if (year > 2012)
        model = year;
        else
            System.out.println("invalid model");
    }
    public String getmaker()
    { return maker;
    }
    public int getmodel()
    { return model;
    }
}
```

Example (Data Hiding)

```
public class Main
{
    public static void main(String[] args) {
        Car c1;
        c1 = new Car();
        Car c2 = new Car();
        c1.setmaker("Honda");
        c1.setmodel(2016);
        c2.setmaker("Toyota");
        c2.setmodel (2008);
        System.out.println(c1.getmaker());
    }
}
```

invalid model
Honda